

MAINCRAFTS TECHNOLOGY

Cybersecurity & Ethical Hacking Internship

VULNERABILITY ASSESSMENT REPORT

Task-3: Vulnerability Scanning & Assessment

Target System: OWASP Juice Shop (Docker) — Internal Lab Environment
Target IP: 192.168.56.102:3000

Prepared By	Vaibhav
Role	Cybersecurity & Ethical Hacking Intern
Intern ID	MT5902
Organization	Maincrafts Technology
Assessment Date	06 July 2026
Classification	Internal / Lab Use Only

1. Executive Summary

This report presents the findings of a vulnerability assessment conducted against an intentionally vulnerable web application (OWASP Juice Shop) hosted within an isolated, Host-Only virtual lab environment as part of Task-3 of the Maincrafts Technology Cybersecurity & Ethical Hacking Internship. The purpose of this assessment was to identify exposed services, misconfigurations, and information-disclosure weaknesses using industry-standard reconnaissance and scanning tools, without performing active exploitation.

The assessment identified a single exposed service (a Node.js web application on port 3000) and confirmed several real, verifiable misconfigurations, including an open directory listing exposing sensitive backup and credential-related files, publicly accessible internal operational metrics, missing HTTP security headers, and a permissive Cross-Origin Resource Sharing (CORS) policy. No critical remote-code-execution vulnerabilities were identified during this phase, as exploitation was explicitly out of scope for Task-3.

A summary of findings by risk rating is provided below. Full technical detail, evidence, and remediation guidance for each finding follows in Section 4.

Risk Rating	Count	Examples
High	1	Open directory listing on /ftp exposing sensitive files
Medium	3	Exposed /metrics endpoint, missing security headers, permissive CORS
Low	2	Information disclosure via /stats.json, X-Recruiting header leak

2. Methodology

This assessment followed a structured, professional vulnerability-assessment workflow, moving from passive discovery to progressively more detailed enumeration. All activity was conducted from a Kali Linux attacker VM against a single target VM (Ubuntu Server hosting OWASP Juice Shop in Docker), both connected via a VirtualBox Host-Only network. No systems outside the lab environment were accessed at any point.

2.1 Rules of Engagement

In-Scope	Out-of-Scope
Lab systems only (192.168.56.0/24)	Any system outside the lab network
Passive and active scanning	Exploitation attempts
Web and network layer visible issues	System destruction or denial-of-service
Manual verification of scanner findings	Real-world / production websites

2.2 Tools Used

Tool	Purpose	Version
Nmap	Host discovery, port and service/version enumeration	7.99

Tool	Purpose	Version
Nikto	Web server misconfiguration and header analysis	2.6.0
Dirsearch	Hidden file and directory discovery	0.4.3
Firefox	Manual verification of automated scanner findings	ESR

2.3 Assessment Workflow

- Step 1 — Host Discovery: confirmed target reachability using Nmap ping sweep.
- Step 2 — Port & Service Enumeration: full TCP port scan with service/version detection.
- Step 3 — Web Vulnerability Scan: Nikto scan against the identified web service.
- Step 4 — Hidden Endpoint Discovery: Dirsearch brute-force scan, followed by manual browser verification of all positive hits.
- Step 5 — Documentation: findings consolidated into this formal Vulnerability Assessment Report.

3. Network & Service Overview

Target IP Address: 192.168.56.102

Network: 192.168.56.0/24 (VirtualBox Host-Only Adapter)

Attacker Host: Kali Linux 2026.1 (VirtualBox)

Open Ports Identified: 1 (3000/tcp)

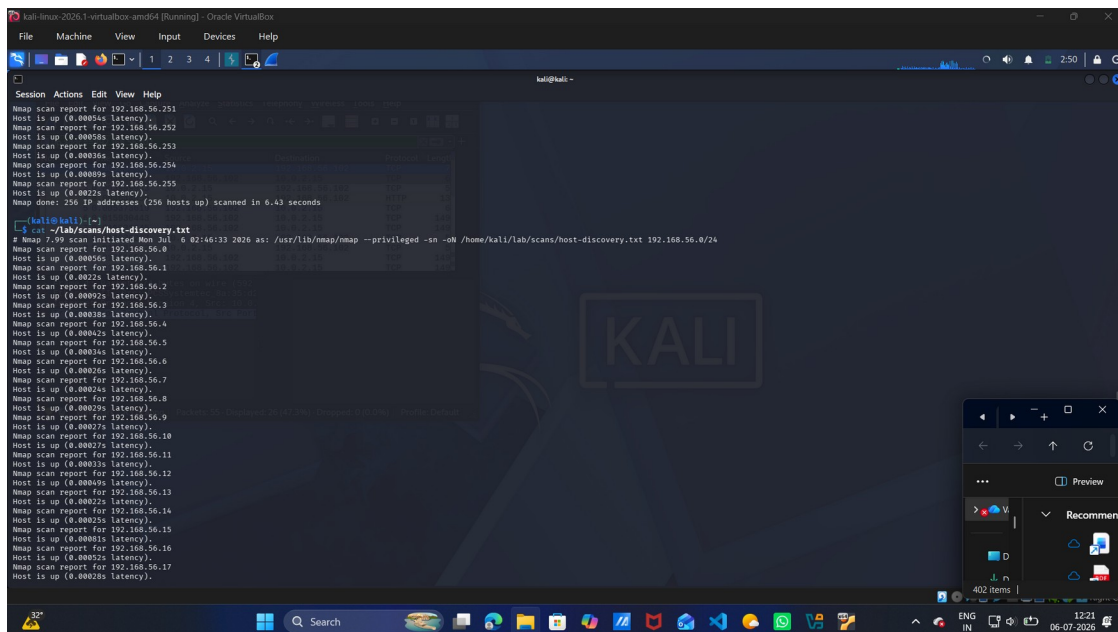


Figure 3.1 — Nmap host discovery confirming target 192.168.56.102 is live on the lab network.

Port	State	Service	Details
3000/tcp	Open	HTTP (Node.js / Express)	Identified as OWASP Juice Shop via HTTP

Port	State	Service	Details
			response body and headers

All other 65,534 TCP ports were reported as filtered with no response, indicating a minimal, well-contained attack surface at the network layer. The single exposed port serves the target web application under assessment.

Notably, the raw HTTP service banner returned by Nmap already revealed several points of interest later confirmed independently by Nikto and manual review: a wildcard CORS header (Access-Control-Allow-Origin: *), a custom X-Recruiting header disclosing an internal path (/#/jobs), and the presence of X-Content-Type-Options and X-Frame-Options headers (both correctly configured).

4. Technical Findings

Each finding below was independently verified by manual browser inspection following automated scanning, in line with professional assessment practice. Findings that appeared only as unconfirmed scanner guesses (for example, generic 500-error responses returned by Dirsearch for numerous /api/ and /rest/ paths) are excluded from this section, as they do not represent confirmed vulnerabilities.

F-01 Open Directory Listing on /ftp Exposing Sensitive Files

Severity	High
Affected Target	http://192.168.56.102:3000/ftp

Description

The /ftp directory was found to be publicly accessible and returns a full directory listing rather than a 403/404 response. The listing exposes multiple files, including backup files (package.json.bak, package-lock.json.bak, coupons_2013.md.bak), a KeePass password database file (incident-support.kdbx), and internal-sounding documents (acquisitions.md, announcement_encrypted.md, legal.md, suspicious_errors.yml). This finding was first flagged as a possible path by Nikto and confirmed as a real, browsable directory during manual verification.

Evidence

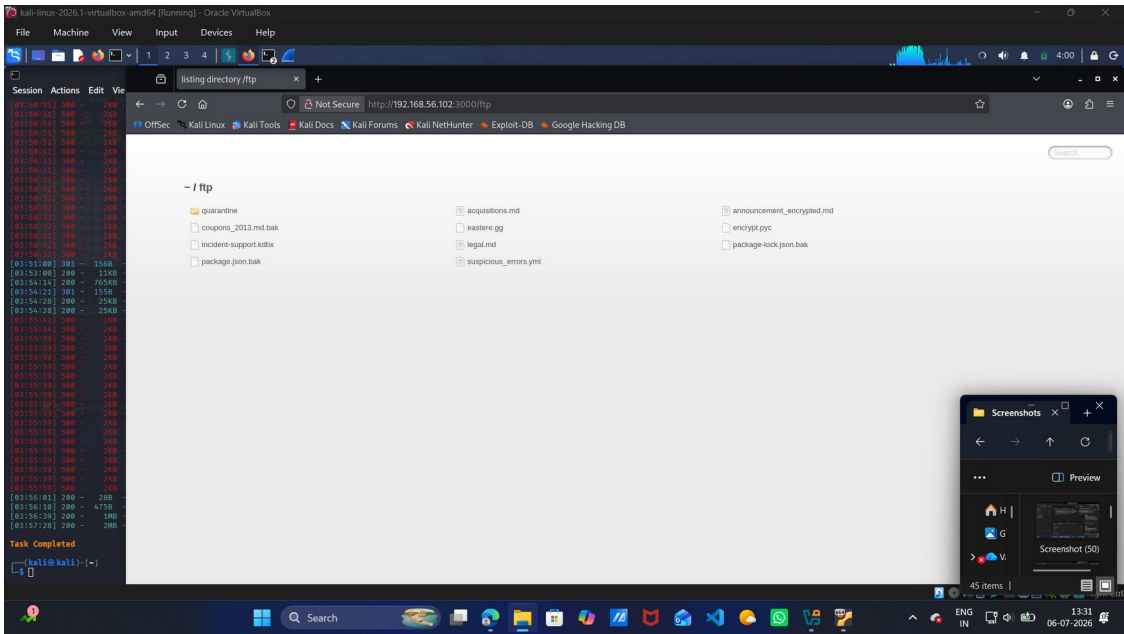


Figure 4.1 — Directory listing of /ftp exposing backup and credential-related files.

Impact

Directory listings allow an attacker to enumerate files that were never intended to be linked publicly. Backup files often contain outdated but still-valid credentials or configuration secrets, and a discoverable .kdbx (password database) file is a direct credential-harvesting target if it can be downloaded and brute-forced offline. At minimum, this exposes internal file-naming conventions and business-sensitive document titles to any unauthenticated visitor.

Remediation

Disable directory listing on the web server / static file middleware (e.g. disable Express 'serve-index' or equivalent). Remove backup files (*.bak) and credential stores (*.kdbx) from any web-accessible directory entirely; these should never reside within the application's served file tree. Apply the principle of least exposure: only files explicitly required by the application should be reachable via HTTP.

F-02 Internal Application Metrics Publicly Exposed (/metrics)

Severity	Medium
Affected Target	http://192.168.56.102:3000/metrics

Description

The /metrics endpoint is accessible without authentication and returns a Prometheus-style metrics export, including HTTP request counters by status code, CPU usage totals, startup task durations, and internal LLM token-usage counters. This endpoint is intended for internal monitoring systems and should not be exposed to unauthenticated, external clients.

Evidence

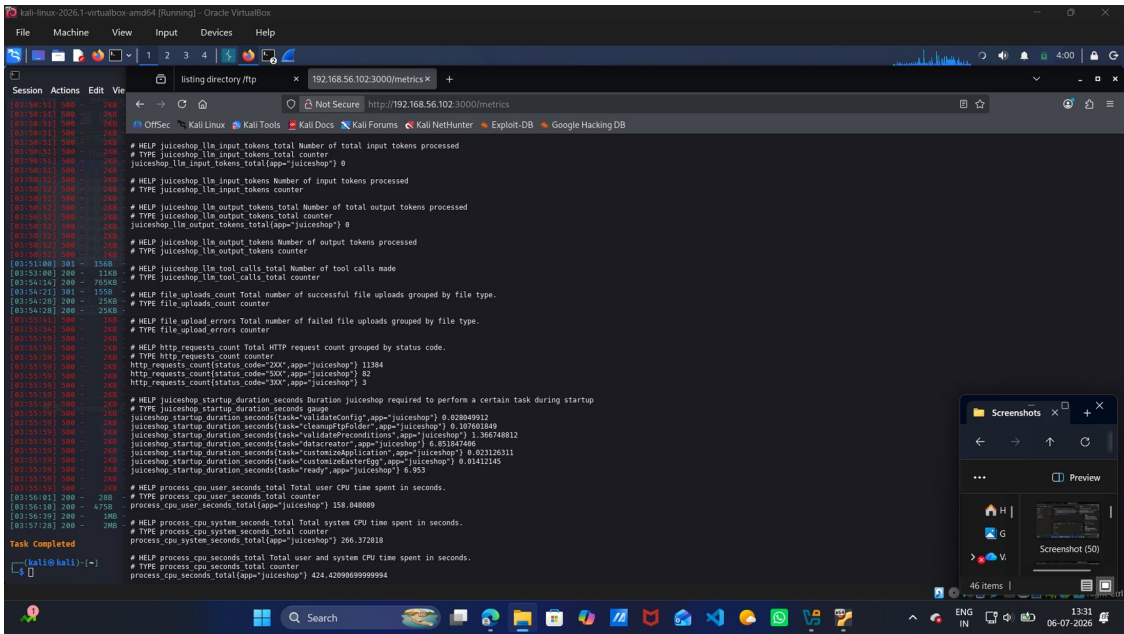


Figure 4.2 — Publicly accessible /metrics endpoint exposing internal operational telemetry.

Impact

Exposed metrics endpoints let an attacker profile the application's internal behaviour: request volume, error rates, and processing characteristics. This aids reconnaissance for timing-based attacks and can reveal the presence of internal features (such as LLM/tool-call integrations) not otherwise advertised, expanding the attacker's understanding of the target beyond what the front-end UI reveals.

Remediation

Restrict /metrics to internal-only network access (e.g. bind to localhost or an internal monitoring VLAN, or require authentication/IP allow-listing at the reverse proxy). Never expose Prometheus-style metrics endpoints on the same public listener as the application itself.

F-03 Missing HTTP Security Headers

Severity	Medium
Affected Target	http://192.168.56.102:3000/

Description

Nikto identified that the application response does not include several recommended security headers: Strict-Transport-Security, Content-Security-Policy, Permissions-Policy, and Referrer-Policy. X-Content-Type-Options and X-Frame-Options were present and correctly configured.

Evidence

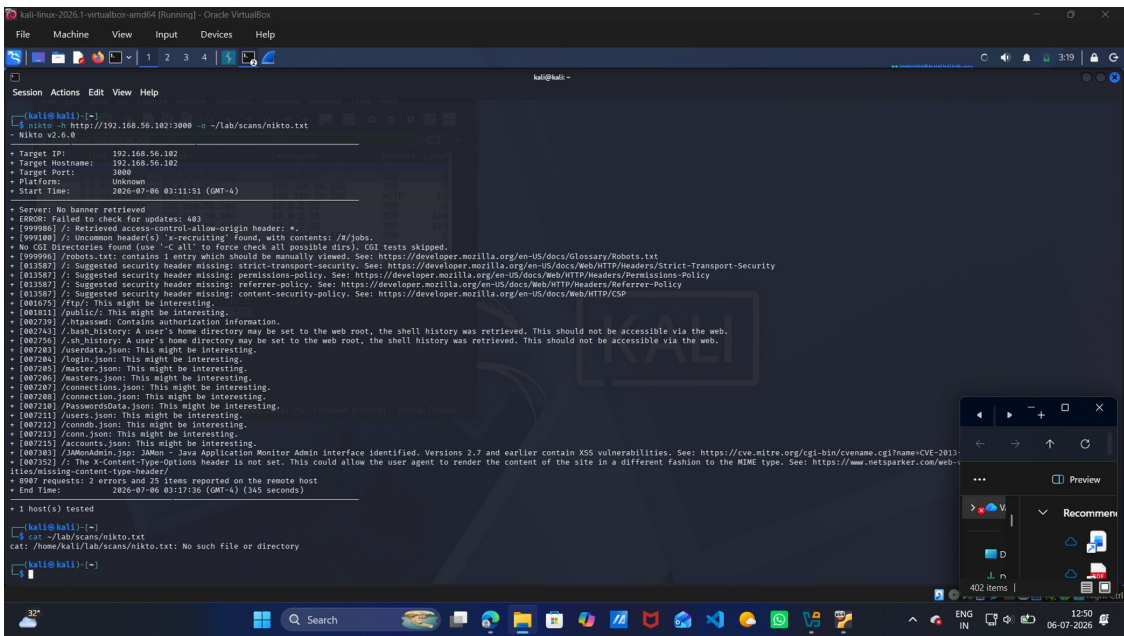


Figure 4.3 — Nikto scan output listing missing security headers and related findings.

Impact

The absence of a Content-Security-Policy increases the impact of any cross-site scripting (XSS) vulnerability that may exist elsewhere in the application, since the browser has no restriction on which script sources it will execute. Missing Strict-Transport-Security means that if the application is ever served over HTTPS in another environment, browsers will not enforce secure connections by default, leaving room for downgrade attacks.

Remediation

Add the missing headers at the application or reverse-proxy layer: a restrictive Content-Security-Policy, Strict-Transport-Security (once HTTPS is in use), a Referrer-Policy such as 'strict-origin-when-cross-origin', and a Permissions-Policy that disables unused browser features (camera, microphone, geolocation, etc.).

F-04 Permissive Cross-Origin Resource Sharing (CORS) Policy

Severity	Medium
Affected Target	http://192.168.56.102:3000/

Description

The application returns Access-Control-Allow-Origin: * on its responses, permitting any origin on the internet to make cross-origin requests to the API and read the responses via client-side JavaScript. This was confirmed both in the raw Nmap service banner and independently by Nikto.

Impact

A wildcard CORS policy combined with any authenticated, session-based API endpoint could allow a malicious website to read data on behalf of a logged-in victim's browser. Even where the current application does not rely on cookie-based sessions for sensitive actions, a permissive CORS policy is a standing misconfiguration that increases risk as the application evolves.

Remediation

Severity	Low
Affected Target	http://192.168.56.102:3000/stats.json

The `/stats.json` endpoint returns internal Angular/webpack build statistics, including input and output module file paths and bundle names (e.g. `main.js`, `chunk-5K74DZ2F.js`, `node_modules` paths). This file is a build artifact and was not intended for production exposure.

[illegible]

While not directly exploitable on its own, this file helps an attacker map the application's internal module structure, third-party dependency versions, and file layout — useful reconnaissance when searching for known vulnerabilities in specific dependency versions.

Exclude build-statistics files (stats.json and similar webpack/Angular CLI artifacts) from the production deployment bundle; these should be generated only for local debugging and never shipped to a public web root.

Severity	Low
----------	-----

Affected Target	http://192.168.56.102:3000/
------------------------	-----------------------------

Description

Every HTTP response from the application includes a non-standard header, X-Recruiting: /#/jobs, revealing the existence of an internal or promotional endpoint not otherwise linked from the main navigation.

Impact

Low direct security impact, but it is an example of unintended information leakage through custom headers, and reflects a broader pattern worth reviewing: custom headers should be audited to ensure they do not unintentionally disclose internal routes, versions, or organizational information.

Remediation

Remove non-essential custom headers from production responses, or confirm the disclosed endpoint is intentionally public and does not reveal anything sensitive.

5. Evidence Appendix

This appendix contains supporting raw scan outputs collected during the assessment, corresponding to the evidence referenced throughout Section 4.

5.1 Dirsearch — Full Endpoint Enumeration

The Dirsearch scan was re-run at a reduced request rate (10 threads, 0.5s delay) after an initial high-speed scan caused the target's lightweight Docker container to become temporarily unresponsive — itself a minor operational observation about the fragility of the test environment under load, rather than a security finding.

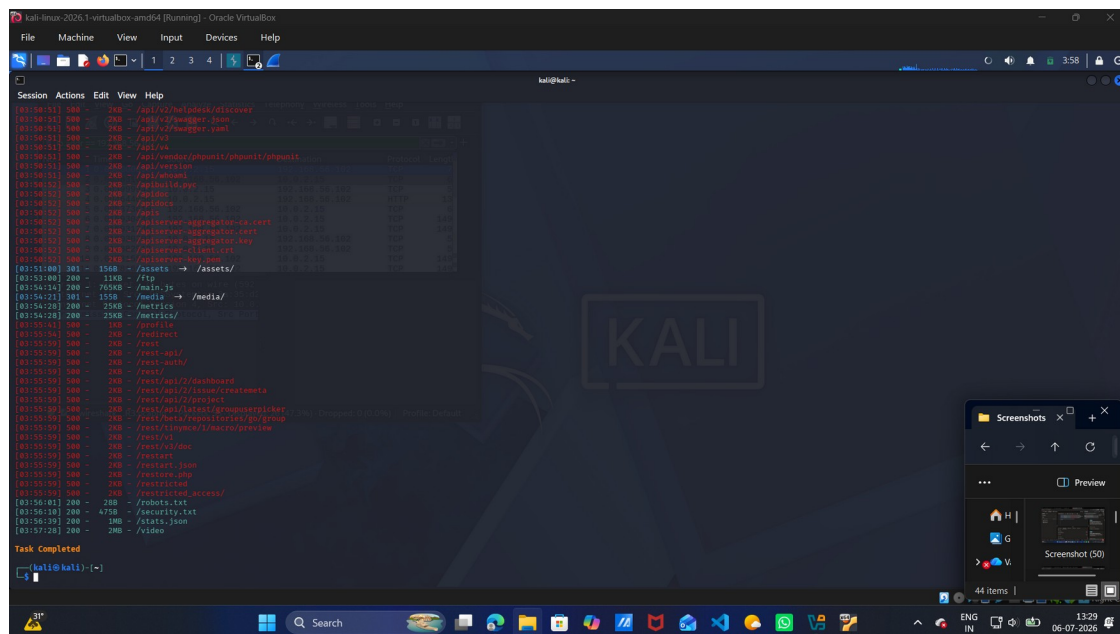


Figure 5.1 — Dirsearch results showing confirmed 200/301 responses (/ftp, /metrics, /stats.json, /main.js, /robots.txt, /security.txt, /video).



6. Conclusion & Next Steps

This assessment successfully identified the target's exposed attack surface and confirmed six real, verifiable findings ranging from Low to High severity, prioritizing manually-verified results over raw automated scanner noise. The target's network-layer exposure is minimal (a single open port), but the web application layer contains meaningful misconfigurations — most notably the open /ftp directory listing exposing sensitive backup and credential files, which should be treated as the highest remediation priority.

No exploitation was performed in this phase, in line with the defined scope of Task-3. The findings documented here — particularly the exposed /ftp listing and the missing security headers — provide natural starting points for Task-4, where deeper web application security testing and controlled exploitation will be carried out to validate real-world impact.

Recommended Priority Order for Remediation

- 1. Disable directory listing and remove sensitive files from /ftp (High).
- 2. Restrict access to /metrics to internal networks only (Medium).
- 3. Implement missing security headers (CSP, HSTS, Permissions-Policy, Referrer-Policy) (Medium).
- 4. Tighten the CORS policy to an explicit origin allow-list (Medium).
- 5. Remove build-artifact files such as stats.json from production (Low).
- 6. Review and remove unnecessary custom headers (Low).